SOFTWARE REQUIREMENTS SPECIFICATION

# Smart Food Redistribution System

AI-Assisted Platform for Surplus Food Donation, Allocation, Pickup and Distribution

**Version 2.0**

| Document Information | Details |
|---|---|
| **Document Version** | 2.0 (supersedes Version 1.0 dated 03 November 2026) |
| **Document Status** | Revised draft for Academic Project Review |
| **Prepared For** | Full Stack Development Course Project |
| **Proposed Technology** | React.js + Django REST Framework + MySQL |
| **AI Scope** | Food Matching, Listing Description, Donation Priority Suggestions (advisory only) |
| **Date** | 04 November 2026 |
| **Revision Focus** | Six areas: status management, quantity allocation, food safety information, volunteer workflow, beneficiary journey and MVP definition |

**Purpose:** This SRS defines the functional and non-functional requirements, user roles, status models, allocation rules, workflows, data requirements, interfaces, constraints and acceptance criteria for the proposed application. It is a project-planning document. Actual food collection, transport and distribution depend on participating donors, NGOs and volunteers. The system is a coordination tool and does not certify food as safe.

# Document Control

| Version | Date | Description | Status |
|---|---|---|---|
| 1.0 | 03-Nov-2026 | Initial SRS for the proposed project scope | Superseded |
| 2.0 | 04-Nov-2026 | Revised after faculty review: separate status fields per entity, quantity allocation and concurrency rules, structured food safety information, detailed volunteer workflow, beneficiary journey and an explicit MVP definition | Draft |

## Summary of Changes from Version 1.0

| Area | Version 1.0 | Version 2.0 change | Section |
|---|---|---|---|
| 1. Status management | One combined status list (Draft to Distributed) on the donation only. | Three independent status fields: Food Donation, Donation Request and Volunteer Assignment, each with its own values, transition rules, owner of change and audit trail. | 4 |
| 2. Quantity allocation | Quantity validated at acceptance; partial quantities, multiple NGOs, cancellation and concurrency not defined. | Total / reserved / available quantity model, partial acceptance, multi-NGO allocation, cancellation and quantity-release rules, row-level locking and conflict handling. | 5 |
| 3. Food safety information | Free-text storage notes and a general disclaimer. | Mandatory structured safety fields, donor declaration, allergen declaration, checkpoints at listing, pickup, receipt and distribution, and a safety-concern report. | 6 |
| 4. Volunteer workflow | Volunteer assigned, then updates pickup and delivery. | Volunteer accept/reject, response timeout, reassignment, pickup check, actual quantity, failure reasons and their effect on request and quantity. | 7 |
| 5. Beneficiary journey | Beneficiaries not represented. | Distribution points, expected beneficiaries, distribution records, leftover accounting, impact summary and feedback to donors, without storing personal data of individual beneficiaries. | 8 |
| 6. MVP definition | Mandatory/optional priorities only. | Release 1 (MVP), Release 2 and Release 3 scope, explicit exclusions, demo scenario and definition of done. | 9 |

**Note on the v1.0 status "Request Pending":** Because a pending request belongs to the Donation Request entity, it is no longer a Food Donation status. The donor dashboard shows a *pending requests* count badge on the listing instead.

# Table of Contents

**Document Control** . . . . . 2

Summary of Changes from Version 1.0 . . . . . 2

**Table of Contents** . . . . . 3

**1. Introduction** . . . . . 6

1.1 Purpose . . . . . 6

1.2 Intended Audience . . . . . 6

1.3 Project Objective . . . . . 6

1.4 Definitions . . . . . 6

**2. Overall Description** . . . . . 6

2.1 Product Perspective . . . . . 6

2.2 Operating Environment . . . . . 6

2.3 Design Principles . . . . . 7

**3. User Classes and Roles** . . . . . 7

**4. Status Management (Area 1)** . . . . . 8

4.1 Principle: One Status Field per Entity . . . . . 8

4.2 Food Donation Status . . . . . 8

4.3 Donation Request Status . . . . . 9

4.4 Volunteer Assignment Status . . . . . 10

4.5 Reason Codes . . . . . 10

4.6 Cross-Entity Rules . . . . . 10

4.7 Status History and Audit . . . . . 11

**5. Quantity Allocation and Concurrency (Area 2)** . . . . . 12

5.1 Quantity Model . . . . . 12

5.2 Design Questions and Decisions . . . . . 12

5.3 Worked Example . . . . . 12

5.4 Allocation Rules . . . . . 13

5.5 Concurrency Control . . . . . 13

**6. Food Safety Information (Area 3)** . . . . . 15

6.1 Principle . . . . . 15

6.2 Mandatory Listing Fields . . . . . 15

6.3 Validation Rules . . . . . 15

6.4 Safety Checkpoints . . . . . 16

6.5 Safety Concern Report . . . . . 16

**7. Volunteer Workflow (Area 4) . . . 17**
7.1 Volunteer Model . . . 17
7.2 Step-by-Step Workflow . . . 17
7.3 Assignment Rules . . . 17
7.4 Failure Handling . . . 17
7.5 Notifications . . . 18

**8. Beneficiary Journey (Area 5) . . . 19**
8.1 Scope . . . 19
8.2 Journey Stages . . . 19
8.3 Distribution Rules . . . 19
8.4 Impact Metrics . . . 19

**9. MVP Definition (Area 6) . . . 20**
9.1 MVP Goal . . . 20
9.2 Release Scope . . . 20
9.3 Explicitly Out of the MVP . . . 20
9.4 MVP Demonstration Scenario . . . 20
9.5 MVP Definition of Done . . . 21
9.6 Indicative Delivery Milestones . . . 21

**10. Functional Requirements . . . 22**

**11. AI-Assisted Features . . . 24**
11.1 Smart Food Matching . . . 24
11.2 Description Generator . . . 24
11.3 Donation Priority Suggestions . . . 24
11.4 Failure and Privacy . . . 24

**12. External Interface Requirements . . . 24**
12.1 User Interface . . . 24
12.2 Software Interfaces . . . 25
12.3 Proposed API Groups . . . 25

**13. Data Requirements and Proposed Database Entities . . . 26**
13.1 Entities . . . 26
13.2 Status Field Codes . . . 26
13.3 Core Relationships . . . 26
13.4 Data Integrity . . . 27

**14. Non-Functional Requirements . . . 27**

**15. Business Rules Summary . . . 27**

**16. Acceptance Criteria and Testing . . . . . 29**

16.1 Suggested Testing Types . . . . . 29

**17. Assumptions, Dependencies and Constraints . . . . . 29**

17.1 Assumptions . . . . . 29

17.2 Dependencies . . . . . 30

17.3 Constraints . . . . . 30

**18. Out of Scope and Future Enhancements . . . . . 30**

**19. Glossary . . . . . 30**

# 1. Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) describes the requirements for the Smart Food Redistribution System. The application connects food donors (restaurants, hotels, caterers and event organisers) with verified non-governmental organisations (NGOs) and their volunteers so that suitable surplus food can be offered, requested, allocated, collected, distributed to beneficiaries and recorded.

## 1.2 Intended Audience

The project instructor, student developer(s), testers and stakeholders who review or use the proposed system.

## 1.3 Project Objective

Reduce avoidable food wastage and improve coordination of surplus-food redistribution through a centralised web application that provides donation listings with structured safety information, quantity-aware NGO requests, volunteer assignments, status tracking, distribution records and administrative monitoring. AI-assisted features provide recommendations only; people remain responsible for verification, food-safety judgement and operational decisions.

## 1.4 Definitions

| Term | Meaning |
|---|---|
| **Food Donor** | A registered and verified restaurant, hotel, caterer or event organiser listing surplus food. |
| **NGO** | A registered and verified organisation that requests food and coordinates distribution to beneficiaries. |
| **Volunteer** | A person affiliated with an NGO who collects or helps deliver a donation. |
| **Beneficiary** | A person or group that finally receives food (for example residents of a shelter). Beneficiaries are not system users in the MVP. |
| **Distribution Point** | A place managed by an NGO where food is handed over to beneficiaries (shelter, community kitchen, school, etc.). |
| **Donation** | A listing describing surplus food available for collection. It acts as a pool of quantity. |
| **Total quantity (T)** | The quantity the donor offers in the listing. |
| **Reserved quantity (R)** | The sum of accepted quantities of all requests in Accepted or Partially Accepted status. |
| **Available quantity (A)** | T minus R: the quantity that can still be accepted for new requests. |
| **Allocation** | The accepted quantity assigned to one NGO request from one donation. |
| **AI suggestion** | A machine-generated recommendation that must be reviewed by an authorised user. |

# 2. Overall Description

## 2.1 Product Perspective

The system is a web-based client-server application. React.js provides the user interface. Django REST Framework exposes APIs and implements business rules in a single service layer. MySQL (InnoDB) stores application records. An optional external AI service may be called by the backend for selected assistance features. The application is not a food-delivery company and does not itself guarantee transport or distribution.

## 2.2 Operating Environment

- Modern desktop or mobile web browser with internet connectivity.
- Frontend: React.js with Vite; responsive HTML, CSS and JavaScript interface.
- Backend: Python, Django and Django REST Framework; scheduled jobs through Django management commands (cron or Celery beat).
- Database: MySQL 8 with InnoDB tables (row-level locking and CHECK constraints).
- API testing and development: Postman; source control: Git and GitHub.

### 2.3 Design Principles

- **Role-based access:** users can access only functions and records permitted for their role.
- **Separate status fields:** each entity owns one status field; statuses are never mixed or overloaded.
- **Single source of truth for quantity:** reserved and available quantity are maintained in one place and changed only by the allocation service.
- **Safe under concurrency:** simultaneous actions by donors, NGOs, volunteers, admins and scheduled jobs cannot over-allocate food or corrupt status.
- **Human oversight:** AI output is advisory and cannot approve food safety or final decisions.
- **Traceability:** every status change and important decision records actor, time and reason.
- **Privacy:** only necessary personal data is collected; no personal data of individual beneficiaries.
- **MVP first:** the core donation-to-distribution workflow is delivered before optional enhancements.

## 3. User Classes and Roles

| Role | Description | Main permissions |
|---|---|---|
| **Food Donor** | Restaurant, hotel, caterer or event organiser. | Manage profile; create, edit and withdraw own listings; fill food safety information; view requests; accept (fully or partially) or reject requests; confirm handover; view impact summary. |
| **NGO** | Organisation receiving and coordinating donations. | Manage profile and distribution points; approve own volunteers; browse and request donations; cancel own requests; assign and reassign volunteers; perform receipt check; record and confirm distribution. |
| **Volunteer** | Person affiliated with one NGO. | Set availability; accept or reject assignments; start pickup; record pickup check and actual quantity; record delivery or failure with reason. |
| **Administrator** | Platform operator or moderator. | Verify, reject or suspend donor and NGO accounts; moderate listings; resolve conflicts; manage categories and settings; review audit history, safety reports and operational reports. |
| **Beneficiary (stakeholder)** | End recipient of food. | Not a system user in the MVP. Represented only through distribution points and aggregate counts. |

A guest browsing role is not part of the mandatory scope. Public listings may be considered later, while contact details and exact addresses must remain protected. **Design decision:** a Volunteer is affiliated with exactly one NGO in the MVP; the NGO approves its volunteers, and only that NGO may assign them.

# 4. Status Management (Area 1)

## 4.1 Principle: One Status Field per Entity

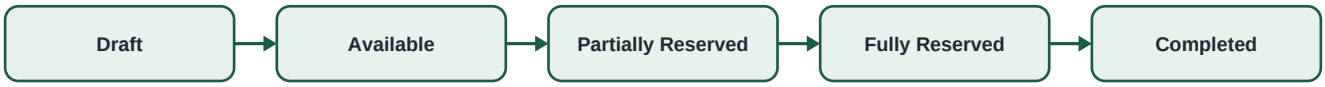
The system maintains three separate status fields. Each answers one question only and changes only through the transitions defined in this section. A change in one entity may trigger a change in another entity only through the cross-entity rules in Section 4.6, executed by the backend service layer inside one database transaction.

| Entity | Status field answers | Changed by | Values |
|---|---|---|---|
| **Food Donation** | What is the state of the listing and its quantity pool? | Donor (publish, withdraw); system (derived from quantities and time); admin (moderation) | Draft, Available, Partially Reserved, Fully Reserved, Completed, Expired, Withdrawn |
| **Donation Request** | What did the donor decide about this NGO's request? | NGO (submit, cancel); donor (decide); system (auto-reject / auto-cancel) | Pending, Accepted, Partially Accepted, Rejected, Cancelled |
| **Volunteer Assignment** | How far has this pickup and delivery task progressed? | NGO (assign); volunteer (progress); system (timeout, deadline) | Assigned, Accepted, Rejected, Pickup In Progress, Picked Up, Delivered, Failed |

> **Rule:** Status values are stored as controlled codes (for example PARTIALLY_RESERVED). The Food Donation status is **derived** by the system from reserved quantity, distribution records and deadline; users cannot set Partially Reserved, Fully Reserved, Completed or Expired directly.

## 4.2 Food Donation Status

**Food Donation lifecycle**




*Exit / terminal states*




| Status | Meaning | Condition (T = total, R = reserved, A = available) |
|---|---|---|
| **Draft** | Donor is preparing the listing; not visible to NGOs. | Not published. Fully editable. |
| **Available** | Published; can receive requests. | R = 0, deadline in the future. |
| **Partially Reserved** | Some quantity accepted; the rest can still be requested. | 0 < R < T. |
| **Fully Reserved** | All quantity is allocated; no new requests accepted. | R = T (including the case where the donor closes the remainder, or the deadline closes it). |
| **Completed** | Food has been distributed and nothing is in progress. | At least one distribution confirmed, no allocation in progress, and either all reserved quantity is distributed or accounted for and the remainder is closed. |
| **Expired** | Pickup deadline passed and nothing was distributed. | Deadline passed, distributed quantity = 0, no allocation in progress. |
| **Withdrawn** | Listing removed by donor (before any allocation) or by admin moderation. | Donor: only when R = 0. Admin: any time, with reason. |

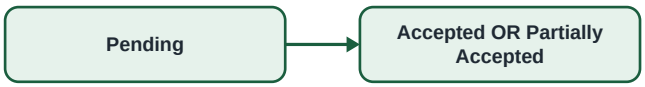
| From | To | Trigger / actor | Conditions and side effects |
|---|---|---|---|
| Draft | Available | Donor publishes | Donor verified; all mandatory fields and safety declaration complete; deadline in future. |
| Draft | Withdrawn | Donor deletes draft | Soft delete only. |
| Available | Partially Reserved | System, on acceptance | 0 < R < T after acceptance. |
| Available / Partially Reserved | Fully Reserved | System, on acceptance | R = T after acceptance. |

| From | To | Trigger / actor | Conditions and side effects |
|---|---|---|---|
| Partially Reserved | Fully Reserved | System at deadline, or donor closes remainder | Unreserved quantity is closed: T is set equal to R. |
| Partially / Fully Reserved | Available | System, on cancellation | Cancellation brings R back to 0 (Section 5.4). |
| Fully Reserved | Partially Reserved | System, on cancellation | R falls below T, before the deadline. |
| Fully Reserved | Completed | System, on distribution confirmation | All allocations distributed or failed; distributed quantity greater than 0. |
| Available / Partially Reserved | Expired | System job at deadline | No accepted allocation remains and nothing was distributed. |
| Available / Partially Reserved | Withdrawn | Donor (R = 0 only) or admin | Pending requests auto-rejected; for admin with accepted requests see Section 4.6. |

Completed, Expired and Withdrawn are terminal: no further transitions and no hard deletion.

## 4.3 Donation Request Status

**Donation Request lifecycle**




*Exit / terminal states*



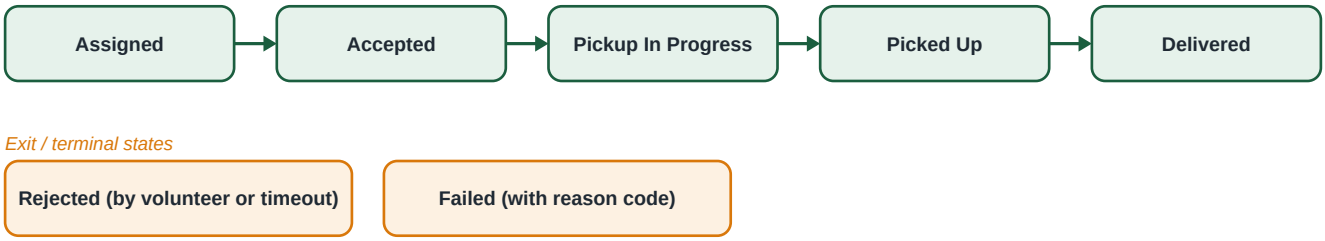

| Status | Meaning | Quantity effect |
|---|---|---|
| **Pending** | NGO submitted the request; awaiting donor decision. | Nothing reserved. |
| **Accepted** | Donor accepted the full requested quantity (accepted = requested). | R increases by accepted quantity. |
| **Partially Accepted** | Donor accepted less than requested (0 < accepted < requested). | R increases by accepted quantity; the unmet part of the request is closed. |
| **Rejected** | Donor rejected, or system auto-rejected with a reason. | None. |
| **Cancelled** | NGO, admin or system cancelled the request, with a reason. | Reserved quantity released only under the rules in Section 5.4. |

| From | To | Trigger / actor | Conditions and side effects |
|---|---|---|---|
| Pending | Accepted | Donor | Accepted quantity = requested quantity and at most A (checked under lock). |
| Pending | Partially Accepted | Donor | Accepted quantity less than requested, at least the NGO's minimum acceptable quantity (if set), at most A. |
| Pending | Rejected | Donor (reason required) or system | System reasons: QUANTITY_UNAVAILABLE, LISTING_WITHDRAWN, LISTING_EXPIRED. |
| Pending | Cancelled | NGO | No quantity effect. |
| Accepted / Partially Accepted | Cancelled | NGO, admin, or system | Allowed until the assignment reaches Picked Up. Reason mandatory. Active assignment is closed as Failed. |

Fulfilment is not a request status. Completion of distribution is recorded by the Distribution Record (Section 8), so a request remains Accepted or Partially Accepted while it is being fulfilled. Rejected and Cancelled are terminal.

## 4.4 Volunteer Assignment Status

**Volunteer Assignment lifecycle**




| Status | Meaning |
|---|---|
| **Assigned** | NGO created the assignment; waiting for the volunteer's response. |
| **Accepted** | Volunteer confirmed that they will do the task. |
| **Rejected** | Volunteer declined or withdrew (reason), or the response timeout expired (system). |
| **Pickup In Progress** | Volunteer has started the trip or is at the donor location. |
| **Picked Up** | Volunteer collected the food, completed the pickup check and recorded actual quantity. |
| **Delivered** | Food reached the NGO or distribution point and the NGO's receipt check was passed. |
| **Failed** | Task could not be completed; reason code recorded (Section 7.4). |

| From | To | Trigger / actor | Conditions and side effects |
|---|---|---|---|
| (new) | Assigned | NGO | Request Accepted or Partially Accepted; no active assignment exists; volunteer belongs to the NGO and is Available; deadline not passed. |
| Assigned | Accepted | Volunteer | Within response timeout. |
| Assigned / Accepted | Rejected | Volunteer or system timeout | Reason required for volunteer action. NGO notified. |
| Accepted | Pickup In Progress | Volunteer | Start trip. |
| Pickup In Progress | Picked Up | Volunteer | Pickup checklist passed; actual quantity recorded; before deadline. |
| Picked Up | Delivered | Volunteer, then NGO receipt check | NGO confirms receipt checklist. |
| Assigned / Accepted / Pickup In Progress | Failed | NGO or system | Request cancelled, deadline missed, food rejected at pickup, donor unavailable, etc. |
| Picked Up | Failed | Volunteer, NGO or admin | For example food rejected on arrival, accident or loss. |

Rejected and Failed end that assignment only. If the request is still Accepted or Partially Accepted, the NGO may create a new assignment; the old row is kept for history. Only one assignment per request may be active at a time.

## 4.5 Reason Codes

| Used for | Reason codes |
|---|---|
| **Request rejection** | DONOR_DECLINED, QUANTITY_UNAVAILABLE, LISTING_WITHDRAWN, LISTING_EXPIRED, FOOD_DETAILS_UNCLEAR |
| **Request cancellation** | NGO_NO_CAPACITY, NGO_CHANGED_PLAN, VOLUNTEER_UNAVAILABLE, DEADLINE_PASSED, FOOD_UNSUITABLE, ADMIN_ACTION, OTHER |
| **Assignment rejection** | VOLUNTEER_BUSY, TOO_FAR, PERSONAL_REASON, NO_RESPONSE_TIMEOUT |
| **Assignment failure** | DONOR_NOT_AVAILABLE, VOLUNTEER_UNREACHABLE, VEHICLE_PROBLEM, FOOD_REJECTED_AT_PICKUP, FOOD_REJECTED_AT_DELIVERY, DEADLINE_MISSED, REQUEST_CANCELLED, OTHER |
| **Listing withdrawal** | DONOR_WITHDREW, FOOD_UNSAFE_REPORTED, POLICY_VIOLATION, ADMIN_ACTION |

## 4.6 Cross-Entity Rules

The table shows how one event changes the three status fields and the quantities. All rows execute atomically.

| Event | Donation | Request | Assignment | Quantity |
|---|---|---|---|---|
| Donor publishes | Draft to Available | - | - | R = 0 |
| NGO submits request | unchanged | created as Pending | - | nothing reserved |
| Donor accepts (full or partial) | to Partially or Fully Reserved | Pending to Accepted / Partially Accepted | - | R increases |
| Donor rejects | unchanged | Pending to Rejected | - | no change |
| A becomes 0 | Fully Reserved | other Pending to Rejected (QUANTITY_UNAVAILABLE) | - | no change |
| NGO assigns volunteer | unchanged | unchanged | created as Assigned | - |
| Volunteer picks up | unchanged | unchanged | to Picked Up | actual quantity stored; no longer returnable |
| Delivery confirmed | unchanged | unchanged | to Delivered | delivered quantity stored |
| NGO confirms distribution | may become Completed | unchanged | unchanged | distributed quantity stored |
| NGO cancels before pickup | recalculated | to Cancelled | active one to Failed (REQUEST_CANCELLED) | R decreases (returned) |
| Deadline passes | to Completed, Expired or Fully Reserved | Pending to Rejected; unpicked Accepted to Cancelled (system) | unpicked active to Failed (DEADLINE_MISSED) | remainder closed; not returned |
| Admin withdraws listing | to Withdrawn | Pending to Rejected; Accepted to Cancelled (ADMIN_ACTION) | active to Failed | not returned |

## 4.7 Status History and Audit

- Every status change of any of the three entities writes one StatusHistory row: entity type, entity id, old status, new status, actor (user id or SYSTEM), reason code, remarks and timestamp.
- Illegal transitions are rejected by the service layer with HTTP 409 and are never written to the database.
- Status, quantity and version fields are read-only in the Django admin site and in all serializers; only the service layer may change them.
- Records are soft-deleted or archived; terminal statuses are never overwritten.

# 5. Quantity Allocation and Concurrency (Area 2)

## 5.1 Quantity Model

A donation is a pool of food measured in one unit (kg, litres, portions or pieces). Requests draw from this pool. The system keeps the following values on each FoodDonation and each DonationRequest.

| Field | Where | Definition |
|---|---|---|
| **total_quantity (T)** | FoodDonation | Quantity offered by the donor. Greater than 0. May be reduced only down to R. |
| **reserved_quantity (R)** | FoodDonation | Sum of accepted_quantity of requests in Accepted or Partially Accepted. Maintained only by the allocation service. |
| **available_quantity (A)** | FoodDonation | T minus R. Shown to NGOs. Computed, not edited. |
| **distributed_quantity (D)** | FoodDonation | Sum of quantities in confirmed Distribution Records (reporting). |
| **requested_quantity** | DonationRequest | Quantity the NGO asks for. At most A at the time of submission. |
| **min_acceptable_quantity** | DonationRequest | Optional. Smallest portion the NGO can use. A partial acceptance below this value is refused. |
| **accepted_quantity** | DonationRequest | Quantity the donor allocated. Zero until decided; at most requested_quantity and at most A when accepted. |
| **actual_quantity** | VolunteerAssignment | Quantity physically collected (recorded at pickup). |

> **Invariants (enforced by service layer and by database CHECK constraints):** 0 <= R <= T; accepted_quantity <= requested_quantity; Pending requests reserve nothing; the sum of accepted quantities of active requests of one donation always equals R.

## 5.2 Design Questions and Decisions

| Question | Decision | Rationale |
|---|---|---|
| Can the donor accept partial quantities? | **Yes.** The donor may accept any quantity q with min_acceptable (if set) <= q <= min(requested, A). The request becomes Partially Accepted; the unmet part is closed and the NGO must submit a new request if it wants more. | Donors often have less than requested or want to serve several NGOs. A minimum acceptable quantity prevents unusable portions. |
| Can multiple NGOs receive food from one donation? | **Yes.** A donation is a pool; each accepted request is a separate allocation with its own volunteer assignment, tracking and distribution record. The sum of allocations never exceeds T. | Maximises use of large surplus (events, caterers) and matches real-world practice. |
| What happens when an NGO cancels after acceptance? | Cancellation is allowed until the assignment reaches Picked Up. A reason is mandatory. The request becomes Cancelled, the active assignment becomes Failed (REQUEST_CANCELLED), and donor and volunteer are notified. After pickup, the request cannot be cancelled; the NGO must use the delivery-failure or receipt-rejection path and the admin is informed. Late cancellations (within a configurable period before the deadline) are flagged and counted on the NGO profile for admin review. | Protects donors and volunteers from last-minute loss and keeps an accountability record. |
| Does cancelled quantity become available again? | **Yes, but only if** cancellation happens before pickup, the pickup deadline has not passed and the donation is not Withdrawn. R decreases by the accepted quantity and the donation status is recalculated. **No** if food was already picked up, if the deadline passed, or if the food was rejected on safety grounds. | Unpicked food is still with the donor and can be reused; food that left the donor or failed a safety check must not be re-offered. |
| What if two donors or admins process requests at the same time? | Every quantity or status change runs in a database transaction that first locks the donation row. The second actor waits, re-reads the current state and either succeeds on the remaining quantity or receives HTTP 409 with the latest state (Section 5.5). | Prevents over-allocation, double acceptance and lost updates. |

## 5.3 Worked Example

Donation of 100 kg. NGO-A requests 60, NGO-B requests 50, NGO-C requests 30.

| Step | Action | R | A | Donation status | Request statuses |
|---|---|---|---|---|---|
| 1 | Donor publishes the listing | 0 | 100 | Available | - |

| Step | Action | R | A | Donation status | Request statuses |
|---|---|---|---|---|---|
| 2 | A, B and C submit requests | 0 | 100 | Available (3 pending requests badge) | A, B, C: Pending |
| 3 | Donor accepts A in full (60) | 60 | 40 | Partially Reserved | A: Accepted |
| 4 | Donor accepts B partially (40 of 50) | 100 | 0 | Fully Reserved | B: Partially Accepted; C auto-rejected (QUANTITY_UNAVAILABLE) |
| 5 | NGO-A cancels before pickup | 40 | 60 | Partially Reserved | A: Cancelled; B unchanged |
| 6 | NGO-C submits a new request for 30; donor accepts | 70 | 30 | Partially Reserved | C: Accepted |
| 7 | B and C complete distribution; deadline passes | 70 | closed | Completed | B, C unchanged; D = 70; remaining 30 closed |

## 5.4 Allocation Rules

- **AL-1 Submit:** A verified NGO may have only one open request (Pending, Accepted or Partially Accepted) per donation. Requested quantity must be positive and at most A at submission time.
- **AL-2 Accept:** Only the owning donor (or an admin acting with a recorded reason) can decide a request. Accepted quantity is validated against A *after* the row lock is taken.
- **AL-3 Auto-reject:** When A becomes 0, all other Pending requests are rejected by the system with QUANTITY_UNAVAILABLE and the NGOs are notified. They may submit a new request if quantity returns.
- **AL-4 Release:** Quantity returns to the pool only when a request in Accepted/Partially Accepted is cancelled while its assignment (if any) is not yet Picked Up and the deadline has not passed.
- **AL-5 Close remainder:** The donor may close the unreserved remainder (T becomes R). The system does this automatically at the pickup deadline.
- **AL-6 Edit:** Total quantity may be changed only to a value at least equal to R. Material fields (category, allergens, consume-by time, location, deadline) are locked once any request is Accepted or Partially Accepted; edits before that notify NGOs with Pending requests.
- **AL-7 Shortfall at pickup:** If actual quantity is less than accepted quantity, the request's accepted quantity is corrected with a recorded reason. The difference is not automatically re-offered.
- **AL-8 Precision:** Quantities are stored as DECIMAL(10,2); units cannot change after the first request.

## 5.5 Concurrency Control

- **C-1 Single entry point:** Accept, reject, cancel, withdraw, expire and admin-override operations go through one service function per action. Views and the Django admin site never modify status or quantity fields directly.
- **C-2 Transaction and lock:** Each operation runs in `transaction.atomic()` and takes `select_for_update()` on the FoodDonation row first, then on the affected request and assignment rows. The fixed order donation, request, assignment avoids deadlocks.
- **C-3 Re-validate after locking:** Current statuses, A and the deadline are re-read inside the lock. Decisions made from a stale screen are refused.
- **C-4 Optimistic version:** Requests and donations carry an integer *version*. The client sends the version it saw; a mismatch returns HTTP 409 with the latest state. The version increments on every change.
- **C-5 Database safety net:** CHECK (reserved_quantity >= 0 AND reserved_quantity <= total_quantity); one active assignment per request through a nullable *active_slot* column with a UNIQUE index on (request_id, active_slot).
- **C-6 After-commit effects:** Notifications and logs are sent after the transaction commits (`transaction.on_commit`), so rolled-back actions never notify anyone.
- **C-7 Idempotency:** Repeating an action (double click, retry) never reserves quantity twice; it returns the same result or HTTP 409 "already processed".
- **C-8 Deadlock retry:** The service retries a bounded number of times (for example 3) when MySQL reports a deadlock.
- **C-9 Scheduled jobs:** The expiry job uses the same services and locks, runs as actor SYSTEM, and processes records in small batches.

| Scenario | Expected behaviour |
|---|---|
| Donor and admin both accept the same Pending request | First commit wins. The second gets HTTP 409 "Request already processed"; history shows one decision. |
| Donor accepts requests X (60) and Y (60) at the same moment on a 100 kg donation | Operations are serialised. The first succeeds; the second sees A = 40 and receives HTTP 409 with the message "Only 40 available" and the option to accept 40 (partial). |
| Donor accepts while NGO cancels the same Pending request | If the cancel commits first the accept fails (request not Pending). If the accept commits first, the cancel then runs as an Accepted-request cancellation and releases the quantity. |
| Donor withdraws the listing while an NGO submits a request | Submit takes the donation lock and finds Withdrawn: HTTP 409; no request is created. |
| Two NGO staff assign different volunteers to the same request | The unique active-assignment constraint plus the request lock allow one; the other gets HTTP 409. |
| Expiry job runs while a volunteer records pickup | Serialised by the lock. If pickup commits first the allocation continues; otherwise the assignment is Failed (DEADLINE_MISSED) and the volunteer sees "Assignment closed". |
| Admin moderation and donor edit at the same time | Same donation lock; the loser re-reads state and is refused if the listing is now Withdrawn. |

# 6. Food Safety Information (Area 3)

## 6.1 Principle

Food redistribution carries public-health risk. The system is a coordination tool: it collects and displays structured safety information declared by humans and prompts human checks at each handover. It never certifies, scores or predicts food safety, and AI must not be used to declare food safe or extend shelf life.

## 6.2 Mandatory Listing Fields

| Field | Values / format | Required | Purpose |
|---|---|---|---|
| Food type (diet) | Vegetarian, Non-vegetarian, Egg, Vegan | Always | Dietary suitability for beneficiaries. |
| Food state | Cooked meal, Packaged, Raw produce, Bakery, Beverage | Always | Selects which other fields apply. |
| Prepared at | Date and time, not in the future | Cooked, Bakery | Time since preparation shown to NGO. |
| Consume-by time | Date and time declared by donor | Cooked, Bakery | Latest time the donor declares the food suitable to eat. |
| Best-before / expiry date | Date from the package label | Packaged | Prevents distribution of expired packaged food. |
| Pickup deadline | Date and time, not later than consume-by | Always | Collection window. |
| Storage condition | Hot-held, Ambient, Refrigerated, Frozen | Always | Tells volunteer how to carry the food. |
| Packaging | Sealed container, Covered tray, Foil/wrap, Original sealed pack, Open | Always | Checked at pickup. |
| Allergen declaration | Multi-select: milk, egg, gluten (wheat), peanuts, tree nuts, soy, fish, shellfish, sesame; or "None declared" or "Unknown" | Always (cannot be blank) | Shown as badges. Unknown is displayed as a warning, never as "allergen-free". |
| Ingredients / handling notes | Free text, optional | Optional | Reheating, serving or handling advice. |
| Temperature at packing | Number in degrees Celsius, optional | Optional | Supports the pickup check. |
| Donor declaration | Checkbox with text: food was prepared and stored hygienically, is fit for consumption and details are accurate; stores user and timestamp | Always (to publish) | Accountability and legal record. |

## 6.3 Validation Rules

- Prepared-at must not be in the future; consume-by must be later than the listing time; pickup deadline must not be later than consume-by.
- Maximum age of cooked food at listing time (hours since preparation) is a configurable admin setting per food category. Values must be set after reviewing applicable food-safety guidance; the system uses them only to block listings, never to declare food safe.
- A listing cannot be published without a complete allergen declaration and the donor declaration.
- Expired packaged food (best-before earlier than today) cannot be listed.
- Safety fields are displayed prominently on the listing, the NGO request screen, the volunteer task and the receipt checklist, and are locked after the first acceptance (AL-6).

## 6.4 Safety Checkpoints

| # | Checkpoint | Performed by | What is checked | If it fails |
|---|---|---|---|---|
| 1 | Listing declaration | Donor | Safety fields complete; declaration ticked. | Cannot publish. |
| 2 | Request acknowledgement | NGO | NGO ticks that it reviewed allergens, consume-by time and storage condition. | Cannot submit request. |
| 3 | Pickup check | Volunteer | Container sealed and clean; no visible spoilage or odour; condition matches storage type; quantity matches. | Pickup refused; assignment Failed (FOOD_REJECTED_AT_PICKUP); request Cancelled (FOOD_UNSUITABLE); quantity not returned; admin flagged. |
| 4 | Receipt check | NGO | Same checklist; consume-by time still valid. | Assignment Failed (FOOD_REJECTED_AT_DELIVERY); quantity recorded as discarded; admin flagged. |
| 5 | Distribution record | NGO | Distributed and discarded quantities accounted. | Leftover or discarded quantity recorded with reason. |

## 6.5 Safety Concern Report

Any participant can report a concern (for example suspected spoilage or an allergic reaction) against a donation. A SafetyReport record links the donation, reporter, description and time. The admin can immediately withdraw the listing (reason FOOD_UNSAFE_REPORTED), suspend the donor or NGO, and review the history. Repeated pickup or receipt rejections for the same donor are highlighted on the admin dashboard.

> **Disclaimer:** Admin verification of an organisation is not food-safety inspection or regulatory approval. Applicable local laws and food-safety requirements in India, including FSSAI requirements on food recovery and distribution, must be reviewed by the project team before any real-world deployment.

# 7. Volunteer Workflow (Area 4)

## 7.1 Volunteer Model

- A volunteer registers with a phone number and service area and is linked to one NGO. The NGO approves the link.
- Volunteer availability: Available, Busy, Offline. Only Available volunteers can be assigned.
- An NGO staff member who collects food personally is created as a volunteer of that NGO.
- A volunteer sees only tasks assigned to them. Donor address and contact are revealed only after the volunteer accepts; NGO contact is revealed for delivery.

## 7.2 Step-by-Step Workflow

| Step | Actor | Action | Result |
|---|---|---|---|
| 1 | NGO | Selects an Accepted or Partially Accepted request and chooses a volunteer. | Assignment = Assigned; volunteer notified; response timer starts. |
| 2 | Volunteer | Reviews quantity, timing, safety information and distance; accepts or rejects (reason). | Accepted, or Rejected (NGO notified to reassign). |
| 3 | System | No response within the response timeout (configurable, default 30 minutes). | Assignment = Rejected (NO_RESPONSE_TIMEOUT); NGO notified. |
| 4 | Volunteer | Taps "Start pickup". | Pickup In Progress; donor notified. |
| 5 | Volunteer | At the donor: completes pickup checklist, records actual quantity, optional photo and note. | Picked Up; request can no longer be cancelled; quantity no longer returnable. |
| 6 | Volunteer | Delivers to NGO or distribution point and marks delivery. | Delivery awaiting NGO receipt check. |
| 7 | NGO | Performs receipt check and confirms. | Delivered; NGO may now record distribution (Section 8). |
| 8 | Any | Problem at any step. | Failed with a reason code (Section 7.4). |

## 7.3 Assignment Rules

- Only one active assignment per request. Reassignment creates a new row; earlier rows remain for history.
- The volunteer must belong to the assigning NGO and be Available; a volunteer cannot hold two overlapping active assignments in the MVP.
- Pickup can be recorded only before the pickup deadline and, if set, after the earliest pickup time.
- Actual quantity may not exceed accepted quantity. A shortfall requires a reason and updates accepted quantity (AL-7).
- Each progress update stores timestamp and user in PickupTracking and in StatusHistory.
- Location is typed or selected by users in the MVP; live GPS tracking is not required.

## 7.4 Failure Handling

| Reason code | Assignment | Request | Quantity | Next step |
|---|---|---|---|---|
| VOLUNTEER_UNREACHABLE, VEHICLE_PROBLEM | Failed | stays Accepted / Partially Accepted | stays reserved | NGO reassigns another volunteer. |
| DONOR_NOT_AVAILABLE | Failed | stays | stays reserved | NGO reassigns or cancels; repeated cases go to admin. |
| FOOD_REJECTED_AT_PICKUP | Failed | Cancelled (FOOD_UNSUITABLE) | not returned | Admin flagged; safety report created. |
| FOOD_REJECTED_AT_DELIVERY | Failed (from Picked Up) | stays; closed by NGO | recorded as discarded | Admin flagged; NGO records reason. |
| DEADLINE_MISSED | Failed (system) | Cancelled (system, DEADLINE_PASSED) | not returned (expired) | None. |
| REQUEST_CANCELLED | Failed (system) | Cancelled by NGO | returned if before pickup and deadline | Volunteer and donor notified. |

## 7.5 Notifications

Notifications are in-app. Donor: new request, pickup started, pickup completed, cancellation. NGO: volunteer response, every assignment status change, failure, delivery. Volunteer: new assignment, cancellation, deadline reminder. Admin: failures with food-safety reasons, repeated late cancellations, safety reports.

# 8. Beneficiary Journey (Area 5)

## 8.1 Scope

Beneficiaries are the people who finally eat the food. In the MVP they are not users and the system stores no names, identity numbers or health data of individual beneficiaries. They are represented by the NGO's distribution points and by aggregate counts that the NGO enters.

## 8.2 Journey Stages

| Stage | Beneficiary experience | System support | Data captured |
|---|---|---|---|
| 1. Need identified | A shelter, school or community has a regular or one-time need for meals. | NGO maintains Distribution Points with type, address, usual headcount and dietary notes. | DistributionPoint |
| 2. Matching and request | NGO selects food that suits the group's diet and size. | Request form asks for distribution point and expected number of beneficiaries; matching uses quantity fit and food type. | request.distribution_point_id, expected_beneficiaries |
| 3. Receiving food | Food arrives at the distribution point. | Volunteer delivers to the chosen point; NGO receipt check. | Assignment Delivered |
| 4. Distribution | Beneficiaries are served. | NGO records one or more distribution sessions: time, beneficiaries served, quantity distributed, quantity discarded or left over (with reason), optional photo and remarks. | DistributionRecord |
| 5. Confirmation | - | NGO confirms completion when distributed plus discarded quantity equals delivered quantity (within a configurable tolerance). | confirmed_at, confirmed_by |
| 6. Impact and feedback | Donor and NGO see what the food achieved. | Donor sees quantity redistributed and beneficiaries served; NGO may send a thank-you note or feedback (Release 2). | Impact summary; feedback text |

## 8.3 Distribution Rules

- One request may have several distribution records (food served over more than one session).
- Sum of distributed and discarded quantities of a request cannot exceed its delivered quantity.
- Completion is confirmed by the NGO once all delivered quantity is accounted for. This updates D on the donation and may change the donation to Completed (Section 4.2).
- Beneficiary counts are estimates entered by the NGO and are labelled as such in reports.

## 8.4 Impact Metrics

Dashboards show total quantity redistributed, number of beneficiaries served, completed donations, quantity discarded, and the share of listed food that was actually distributed, filterable by date, donor, NGO and category.

**Future beneficiary features (outside MVP):** beneficiary feedback via QR code, recurring need schedules for distribution points, and demand forecasting.

# 9. MVP Definition (Area 6)

## 9.1 MVP Goal

Deliver one complete, correct and demonstrable path: a verified donor lists food with safety information; verified NGOs request it; the donor accepts full or partial quantities without ever over-allocating; the NGO assigns a volunteer who picks up and delivers; the NGO records distribution to beneficiaries; every step is audited. Exception handling that protects quantity and food safety (cancellation, failure, expiry) is part of the MVP; decorative or integration-heavy features are not.

## 9.2 Release Scope

| Feature area | Release 1 (MVP) | Release 2 | Release 3 / Future |
|---|---|---|---|
| Accounts and roles | Register, login, RBAC, admin verification, volunteer-NGO link | Password reset by email | Social login |
| Donation listing | Create, edit, publish, withdraw; structured safety fields and declaration | Image upload with validation | Recurring donors, templates |
| Requests | Request with quantity, optional minimum, distribution point; accept, partial accept, reject, cancel | Waitlist and alerts when quantity returns | - |
| Quantity and concurrency | T / R / A model, locking, 409 conflict handling, auto-reject, expiry job | Returned-quantity alerts | - |
| Statuses and audit | Three status fields, transition rules, StatusHistory | Admin status override screen | - |
| Volunteer workflow | Assign, accept / reject, timeout, reassign, pickup check, delivery, failure reasons | Photo evidence, availability scheduling | Live GPS and route optimisation |
| Beneficiary journey | Distribution points, distribution records, completion and impact counts | Thank-you note and feedback | Beneficiary QR feedback |
| Notifications | In-app notifications | Email notifications | SMS / WhatsApp |
| Dashboards and reports | Role dashboards, count-based reports with filters | CSV / PDF export | Analytics, forecasting |
| Matching, priority, description | Rule-based matching score, rule-based priority, template-based description helper | LLM-generated description via provider API | Learned matching |
| Safety | Declaration, checklists, concern report, admin withdrawal | Incident workflow and statistics | Regulator integration |

## 9.3 Explicitly Out of the MVP

- Live GPS tracking, route optimisation, courier integration.
- Payments, tax receipts, native mobile apps, multilingual interface, voice assistant.
- Image-based food-quality detection (not suitable for safety certification).
- Beneficiary logins, individual beneficiary records, integration with NGO registries or government platforms.
- Large-language-model features are optional in the MVP; the rule-based and template versions satisfy the requirements.

## 9.4 MVP Demonstration Scenario

- 1. Admin verifies Donor D and NGOs N1 and N2; N1 approves volunteer V1.
- 2. D publishes 100 kg of cooked rice and curry with allergen declaration and consume-by time.
- 3. N1 requests 60 kg and N2 requests 50 kg for their distribution points.
- 4. D accepts N1 in full and N2 partially (40 kg). Donation becomes Fully Reserved.
- 5. Two browser sessions try to accept conflicting quantities at once; one receives a clear 409 message.
- 6. N1 assigns V1; V1 accepts, starts pickup, passes the checklist and records actual quantity; then delivers.
- 7. N1 confirms receipt and records distribution of 55 beneficiaries served and 1 kg discarded.
- 8. N2 cancels before pickup; quantity returns and the donation becomes Partially Reserved.
- 9. The expiry job runs after the deadline; the donation becomes Completed.

- 10. Admin reviews the StatusHistory and the dashboard counts.

## 9.5 MVP Definition of Done

- All Must requirements marked R1 in Section 10 are implemented and pass their acceptance criteria (Section 16).
- Concurrency tests demonstrate no over-allocation under parallel requests.
- Every illegal status transition is rejected by an automated test.
- Role and object-level permission tests pass.
- The demonstration scenario runs end-to-end on seeded data without manual database changes.
- AI or provider failure does not block any core workflow.

## 9.6 Indicative Delivery Milestones

| Milestone | Content |
|---|---|
| **M1 Foundation** | Project setup, database schema, authentication, roles, admin verification. |
| **M2 Donation and safety** | Listing CRUD, safety fields and declaration, listing status logic, search and filters. |
| **M3 Requests and allocation** | Requests, accept / partial / reject, quantity model, locking, conflict handling, cancellation rules. |
| **M4 Volunteer and distribution** | Assignments, tracking, checklists, failure handling, distribution records. |
| **M5 Dashboards, rules-based AI, hardening** | Notifications, dashboards, matching and priority rules, expiry job, testing and demo data. |

# 10. Functional Requirements

Priority uses Must / Should / Could. Release indicates when the requirement is delivered (R1 = MVP).

| ID | Requirement | Priority | Rel. |
|---|---|---|---|
| **Accounts and verification** | | | |
| FR-01 | The system shall allow registration as Donor, NGO or Volunteer; admin accounts are created only by an administrator. | Must | R1 |
| FR-02 | The system shall authenticate users and enforce role-based and object-level permissions. | Must | R1 |
| FR-03 | The system shall maintain account status (pending, verified, rejected, suspended); admin shall verify, reject or suspend donors and NGOs; only verified donors and NGOs may take live actions. | Must | R1 |
| FR-04 | The system shall link each volunteer to one NGO with NGO approval and maintain volunteer availability. | Must | R1 |
| FR-05 | NGOs shall manage Distribution Points (name, type, address, usual headcount, dietary notes). | Must | R1 |
| **Donation listing and food safety** | | | |
| FR-06 | Donors shall create drafts, publish, edit and withdraw their own listings subject to the rules in Sections 4.2 and 5.4. | Must | R1 |
| FR-07 | The system shall store category, unit, total quantity, preparation time, pickup window and location for every listing. | Must | R1 |
| FR-08 | The system shall require the structured food safety fields and donor declaration of Section 6.2 before publishing. | Must | R1 |
| FR-09 | The system shall lock material fields once a request is Accepted or Partially Accepted and notify NGOs with Pending requests when a listing is edited. | Should | R1 |
| FR-10 | The system shall support optional food image upload with file-type and size validation. | Should | R2 |
| **Requests and allocation** | | | |
| FR-11 | Verified NGOs shall search and filter Available and Partially Reserved listings by area, category, quantity, food type, allergens and deadline. | Must | R1 |
| FR-12 | NGOs shall submit a request with quantity, optional minimum acceptable quantity, distribution point, expected beneficiaries, notes and safety acknowledgement. | Must | R1 |
| FR-13 | The system shall allow only one open request per NGO per donation. | Must | R1 |
| FR-14 | Donors shall accept in full, accept partially or reject a request with a reason where required. | Must | R1 |
| FR-15 | NGOs shall cancel a Pending request, or an accepted request before pickup, with a mandatory reason. | Must | R1 |
| FR-16 | The system shall maintain total, reserved and available quantity and enforce the invariants of Section 5.1. | Must | R1 |
| FR-17 | The system shall derive and update the Food Donation status automatically from quantities, distribution and deadline. | Must | R1 |
| FR-18 | The system shall release reserved quantity only under rule AL-4 and shall auto-reject Pending requests under rule AL-3. | Must | R1 |
| FR-19 | The system shall execute allocation operations with transactions and row-level locking and shall detect stale actions through a version check, returning HTTP 409 with current state. | Must | R1 |
| **Volunteer workflow** | | | |
| FR-20 | NGOs shall assign and reassign volunteers; only one assignment per request may be active. | Must | R1 |
| FR-21 | Volunteers shall accept or reject assignments; the system shall apply a response timeout. | Must | R1 |
| FR-22 | Volunteers shall update Pickup In Progress, Picked Up, Delivered and Failed with timestamp and reason where applicable. | Must | R1 |
| FR-23 | The system shall present the pickup checklist and record actual picked-up quantity. | Must | R1 |
| FR-24 | The NGO shall complete a receipt checklist before an assignment becomes Delivered. | Must | R1 |
| FR-25 | The system shall show donor and NGO contact details only to authorised parties at the right stage. | Must | R1 |
| **Distribution and beneficiaries** | | | |
| FR-26 | NGOs shall record distribution sessions (beneficiaries served, distributed quantity, discarded quantity and reason, remarks). | Must | R1 |
| FR-27 | NGOs shall confirm distribution completion when quantities are accounted for. | Must | R1 |

| ID | Requirement | Priority | Rel. |
|---|---|---|---|
| FR-28 | Donors shall view an impact summary of their donations. | Should | R1 |
| FR-29 | NGOs shall send thank-you or feedback notes to donors. | Could | R2 |
| **Status, audit and scheduled processing** | | | |
| FR-30 | The system shall keep separate status fields for Food Donation, Donation Request and Volunteer Assignment and enforce the allowed transitions. | Must | R1 |
| FR-31 | The system shall record every status change with actor, old and new status, reason code and timestamp. | Must | R1 |
| FR-32 | A scheduled job shall process deadlines (expire listings, auto-reject, auto-cancel, fail unpicked assignments, assignment timeouts). | Must | R1 |
| **Notifications, dashboards and reports** | | | |
| FR-33 | The system shall provide in-app notifications for key events in Section 7.5. | Should | R1 |
| FR-34 | The system shall provide role-specific dashboards and filtered count-based reports by date, donor, NGO, category and status. | Must | R1 |
| FR-35 | The system shall export reports as CSV or PDF. | Could | R2 |
| **Administration and safety** | | | |
| FR-36 | Admins shall moderate listings with a recorded reason and shall manage categories and configuration values (timeouts, thresholds). | Must | R1 |
| FR-37 | Admins shall review users, donations, requests, assignments, distribution records and status history. | Must | R1 |
| FR-38 | Any participant shall be able to report a food safety concern; admins shall review and act on it. | Should | R1 |
| **AI-assisted features** | | | |
| FR-39 | The system shall rank donations for an NGO with a transparent score and reasons (Section 11.1). | Should | R1 |
| FR-40 | The system shall suggest urgency levels with reasons (Section 11.3). | Should | R1 |
| FR-41 | The system shall offer an editable description suggestion (template in R1, provider-based in R2). | Could | R1/R2 |
| FR-42 | The system shall log AI suggestions and the user's action with data minimisation. | Could | R2 |

# 11. AI-Assisted Features

AI features assist users and reduce coordination effort. Output is advisory, explainable and never replaces human judgement or food-safety checks. The first version uses deterministic rules; a provider-based model is optional.

## 11.1 Smart Food Matching

**Purpose:** recommend suitable listings to an NGO. **Inputs:** NGO category preference, requested quantity, service area, collection window, distribution point headcount and diet; listing category, available quantity, food type, location and deadline. **Output:** ranked list with reasons.

| Factor | Example weight | Reason shown |
|---|---|---|
| Category and food-type match | 30 | Matches your preferred category and diet |
| Quantity fit (A close to need, not far above or below) | 25 | Available quantity suits your need |
| Deadline suitability against collection window | 20 | Pickup deadline fits your window |
| Area / proximity (if location data exists) | 15 | In your service area |
| Distribution point headcount fit | 10 | Suitable for about N people |

Unverified donors, Withdrawn, Expired and Fully Reserved listings are excluded. The ranking must never imply that food is safe, and weights are configurable.

## 11.2 Description Generator

Builds a suggested description from the structured fields entered by the donor (name, category, quantity, preparation time, storage, allergens, notes). The donor edits and approves it before publishing. It must not invent ingredients, allergens, freshness, preparation conditions or certifications. R1 uses a template; R2 may use a provider API.

## 11.3 Donation Priority Suggestions

| Level | Rule (thresholds configurable) | Example reason |
|---|---|---|
| High | Less than 2 hours to pickup deadline and (unreserved, or accepted without active assignment, or assignment not yet accepted) | Pickup deadline is approaching and no volunteer is confirmed |
| Medium | Less than 6 hours to deadline, or a request has been Pending for a long time | Requests are waiting for a decision |
| Low | Otherwise | Enough time remaining |

Priority is an operational reminder only and is never a food-safety assessment.

## 11.4 Failure and Privacy

- If the AI provider is unavailable, all core functions continue; rule-based or template output is used.
- API keys live in environment variables on the server, never in frontend code.
- Only the minimum data needed for the task is sent to a provider; no contact details or personal data.
- Users can review and correct all AI output before use.

# 12. External Interface Requirements

## 12.1 User Interface

- Responsive layouts for desktop, tablet and mobile browsers.
- Separate dashboards for Donor, NGO, Volunteer and Admin.
- Quantity bar on each listing showing Reserved, Available and Distributed.
- Consistent status labels and colour chips; each of the three statuses is displayed with its own label.
- Allergen badges and a consume-by countdown on listings and tasks.
- Field-level validation messages; clear confirmation for accept, reject, cancel, withdraw and deliver actions.
- Conflict dialog that shows the latest state when the server returns HTTP 409.

## 12.2 Software Interfaces

| Interface | Requirement |
|---|---|
| **React to Django API** | JSON REST APIs over HTTP/HTTPS with token-based authentication. |
| **Django REST Framework** | Validate input, enforce permissions, call the service layer, return suitable HTTP codes (400 validation, 403 permission, 404 not found, 409 conflict or stale state). |
| **MySQL** | Persist all records; InnoDB; CHECK and UNIQUE constraints as in Section 5.5. |
| **Scheduler** | Management command run every minute for deadline processing and timeouts. |
| **AI provider API (optional)** | Called only from the backend; provider configurable. |
| **File storage** | Images and evidence stored in media storage; database keeps references. |

## 12.3 Proposed API Groups

| API group | Example operations |
|---|---|
| **/api/auth/** | register, login, logout, token refresh, profile |
| **/api/donors/, /api/ngos/, /api/volunteers/** | profiles, verification status, volunteer approval |
| **/api/distribution-points/** | create, list, update NGO distribution points |
| **/api/donations/** | create, list, detail, update, publish, withdraw, close-remainder, filter |
| **/api/requests/** | create, list, accept (with accepted_quantity and version), reject, cancel |
| **/api/assignments/** | assign, respond (accept / reject), reassign, list tasks |
| **/api/tracking/** | start pickup, pickup check, delivered, failed; status history |
| **/api/distributions/** | record session, confirm completion |
| **/api/safety-reports/** | create report, list, resolve |
| **/api/notifications/** | list, mark read |
| **/api/reports/** | dashboard metrics and filtered reports |
| **/api/ai/** | matching, description, priority |

These are proposed groups, not a final API contract. Exact paths and payloads are finalised during implementation.

# 13. Data Requirements and Proposed Database Entities

## 13.1 Entities

| Entity | Key data (status fields in bold) |
|---|---|
| **User** | id, name, email, password_hash, phone, role, **account_status**, created_at |
| **DonorProfile** | id, user_id, organisation_name, address, location fields, verification_status |
| **NGOProfile** | id, user_id, organisation_name, service_area, address, verification_status, late_cancel_count |
| **VolunteerProfile** | id, user_id, ngo_id, availability_status, service_area, ngo_approved |
| **DistributionPoint** | id, ngo_id, name, type, address, usual_headcount, dietary_notes, is_active |
| **FoodCategory** | id, category_name, description, max_age_hours_policy |
| **FoodDonation** | id, donor_id, category_id, food_name, unit, total_quantity, reserved_quantity, distributed_quantity, food_type, food_state, prepared_at, consume_by, best_before, pickup_deadline, earliest_pickup, storage_condition, packaging, allergens, ingredients_notes, temperature_c, location, image, donor_declared_at, **status**, version, created_at, updated_at |
| **DonationRequest** | id, donation_id, ngo_id, distribution_point_id, requested_quantity, min_acceptable_quantity, accepted_quantity, expected_beneficiaries, ngo_safety_ack_at, message, **status**, decision_reason, cancelled_by, cancel_reason, requested_at, decision_at, version |
| **VolunteerAssignment** | id, request_id, volunteer_id, assigned_by, assigned_at, responded_at, actual_quantity, failure_reason, active_slot, **status** |
| **PickupTracking** | id, assignment_id, event_type, notes, photo, event_time, updated_by |
| **HandoverCheck** | id, assignment_id, stage (PICKUP or RECEIPT), checklist_result, passed, checked_by, checked_at, notes |
| **DistributionRecord** | id, request_id, distribution_point_id, distributed_at, beneficiaries_served, distributed_quantity, discarded_quantity, discard_reason, remarks, photo, confirmed_at, confirmed_by |
| **StatusHistory** | id, entity_type (DONATION, REQUEST, ASSIGNMENT), entity_id, old_status, new_status, changed_by (or SYSTEM), reason_code, remarks, changed_at |
| **SafetyReport** | id, donation_id, reported_by, description, created_at, resolved_by, resolution, resolved_at |
| **Notification** | id, recipient_id, title, message, is_read, created_at |
| **AIRecommendationLog** | id, feature_type, related_record_id, input_summary, output_summary, user_action, created_at |

## 13.2 Status Field Codes

| Field | Allowed values |
|---|---|
| **FoodDonation.status** | DRAFT, AVAILABLE, PARTIALLY_RESERVED, FULLY_RESERVED, COMPLETED, EXPIRED, WITHDRAWN |
| **DonationRequest.status** | PENDING, ACCEPTED, PARTIALLY_ACCEPTED, REJECTED, CANCELLED |
| **VolunteerAssignment.status** | ASSIGNED, ACCEPTED, REJECTED, PICKUP_IN_PROGRESS, PICKED_UP, DELIVERED, FAILED |
| **User.account_status and verification_status** | PENDING, VERIFIED, REJECTED, SUSPENDED |
| **VolunteerProfile.availability_status** | AVAILABLE, BUSY, OFFLINE |

## 13.3 Core Relationships

- One Donor has many FoodDonations; one FoodDonation has many DonationRequests (pool of quantity).
- One NGO has many DonationRequests, DistributionPoints and Volunteers.
- One DonationRequest has many VolunteerAssignments over time but at most one active; one assignment has many PickupTracking events and up to two HandoverChecks.
- One DonationRequest has many DistributionRecords.
- StatusHistory references any of the three status-bearing entities through entity type and id.

### 13.4 Data Integrity

- Quantities are positive DECIMAL(10,2); unit chosen from a fixed list.
- CHECK: reserved_quantity between 0 and total_quantity; accepted_quantity between 0 and requested_quantity.
- UNIQUE (request_id, active_slot) allows one active assignment per request.
- Status transitions validated against Section 4; terminal records are never hard-deleted.
- Foreign keys prevent orphan requests, assignments and distribution records.
- All timestamps stored in UTC and displayed in the user's local time (IST).

## 14. Non-Functional Requirements

| ID | Category | Requirement |
|---|---|---|
| NFR-01 | Security | Passwords hashed with Django's secure hashing; API keys only in server configuration. |
| NFR-02 | Authorisation | Role-based and object-level permissions prevent viewing or changing records outside a user's access. |
| NFR-03 | Privacy | Contact and exact location shown only to authorised parties at the right stage; no personal data of individual beneficiaries. |
| NFR-04 | Data consistency | Quantity and status changes are atomic; no over-allocation under concurrent use (tested with parallel requests). |
| NFR-05 | Performance | Common pages and API calls respond within a few seconds on academic test data; allocation operations hold locks only briefly. |
| NFR-06 | Reliability | Core workflow works when the AI service is unavailable; scheduled job is idempotent and safe to re-run. |
| NFR-07 | Auditability | All status changes and decisions record user, time and reason; history cannot be edited. |
| NFR-08 | Usability | Clear labels, consistent navigation, understandable conflict and validation messages. |
| NFR-09 | Maintainability | Frontend, API layer, service layer (state machine and allocation) and AI integration are separate modules. |
| NFR-10 | Compatibility | Current versions of major browsers. |
| NFR-11 | Scalability | More donors, NGOs and records without changing the core workflow. |

## 15. Business Rules Summary

| ID | Rule |
|---|---|
| BR-01 | Only verified donors may publish and only verified NGOs may request. |
| BR-02 | Pending requests reserve no quantity. |
| BR-03 | Accepted quantity is at most the lesser of requested quantity and available quantity, checked under lock. |
| BR-04 | Reserved quantity is never negative and never above total quantity. |
| BR-05 | An NGO may have only one open request per donation. |
| BR-06 | Cancellation of an accepted request is allowed until the assignment is Picked Up; a reason is mandatory. |
| BR-07 | Quantity returns to the pool only when cancelled before pickup and before the deadline. |
| BR-08 | Only one assignment per request is active at a time; volunteers belong to the assigning NGO. |
| BR-09 | Pickup is recorded only within the pickup window and requires the pickup checklist. |
| BR-10 | A listing needs the full safety fields and donor declaration; allergen field cannot be blank. |
| BR-11 | Food rejected on safety grounds is never returned to the pool and triggers admin review. |
| BR-12 | Distributed plus discarded quantity cannot exceed delivered quantity. |
| BR-13 | AI never decides food safety or makes the final decision. |
| BR-14 | Terminal statuses (Completed, Expired, Withdrawn, Rejected, Cancelled, Delivered, Failed) are immutable; no hard deletes. |

**Food-safety note:** food that is expired, visibly spoiled, contaminated or otherwise considered unsafe must not be listed or distributed. NGOs may reject any donation if suitability is uncertain.

# 16. Acceptance Criteria and Testing

| ID | Acceptance criterion | Related |
|---|---|---|
| AC-01 | A donor can register, be verified and publish a listing only when all safety fields and the declaration are complete. | FR-06 to FR-08 |
| AC-02 | An unverified donor or NGO cannot publish or request. | FR-03 |
| AC-03 | An NGO can search, filter and request an available donation and must acknowledge safety information. | FR-11, FR-12 |
| AC-04 | A donor can accept in full, accept partially or reject; request and donation statuses update correctly. | FR-14, FR-17 |
| AC-05 | Accepted quantities never exceed total quantity; available quantity shown to NGOs is always T minus R. | FR-16 |
| AC-06 | Multiple NGOs can receive food from one donation, each with a separate assignment and distribution record. | FR-14, FR-20 |
| AC-07 | When A reaches 0, other Pending requests are auto-rejected with QUANTITY_UNAVAILABLE. | FR-18 |
| AC-08 | Cancelling before pickup and before the deadline returns quantity and recalculates the donation status. | FR-15, FR-18 |
| AC-09 | Cancelling after pickup is refused; quantity of cancelled-after-deadline or safety-rejected food is not returned. | FR-15, FR-18 |
| AC-10 | Parallel accept operations on the same donation (for example 20 threads) never produce R greater than T; losers get HTTP 409. | FR-19 |
| AC-11 | A stale version submitted by a client is refused with HTTP 409 and current state. | FR-19 |
| AC-12 | Every allowed transition succeeds and every disallowed transition is refused for all three status fields. | FR-30 |
| AC-13 | Each status change creates a StatusHistory row with actor, reason and time. | FR-31 |
| AC-14 | A volunteer can accept or reject; a missed response timeout sets Rejected and notifies the NGO. | FR-21 |
| AC-15 | Pickup requires the checklist and records actual quantity; a failed checklist sets Failed and cancels the request. | FR-22, FR-23 |
| AC-16 | The NGO can reassign after Rejected or Failed; only one assignment is active. | FR-20 |
| AC-17 | The NGO receipt check is required before Delivered; rejection records discarded quantity and flags the admin. | FR-24 |
| AC-18 | Distribution records can be added, accounting rules are enforced, and completion updates the donation to Completed when conditions are met. | FR-26, FR-27 |
| AC-19 | After the deadline the scheduled job expires listings, closes remainders and fails unpicked assignments correctly. | FR-32 |
| AC-20 | Admin can verify, moderate, suspend and review history; moderation cascades as in Section 4.6. | FR-36, FR-37 |
| AC-21 | Dashboard counts and impact figures match saved records. | FR-28, FR-34 |
| AC-22 | AI or description failure does not block core workflows; suggestions are editable and do not claim safety. | FR-39 to FR-41 |
| AC-23 | Role and object-level permissions prevent unauthorised access; contact details appear only at permitted stages. | FR-02, FR-25 |

## 16.1 Suggested Testing Types

- Unit tests for models, serializers, permissions, the state machine and allocation service.
- Concurrency tests using threads or multiple database connections against MySQL (not SQLite) to verify locking.
- API tests with Postman and Django test client, including negative and conflict cases.
- Integration tests across React, DRF and MySQL; end-to-end workflow tests for the demonstration scenario.
- Scheduled-job tests with controlled clock for deadline, timeout and expiry cases.
- AI integration tests for empty input, invalid output, timeout and provider unavailability.

# 17. Assumptions, Dependencies and Constraints

## 17.1 Assumptions

- Participants provide accurate information and have internet access.
- The platform administrator can review and verify donors and NGOs.

- Locations are entered by users; live GPS is not required.
- AI provider access and limits vary; graceful fallback is required.

### 17.2 Dependencies

- Python, Django, Django REST Framework and MySQL 8 environment; React.js and Vite.
- A scheduler (cron or Celery beat) for deadline processing.
- Optional AI provider API and credentials; configured media storage for images.

### 17.3 Constraints

- The first version is an academic web application, not a certified food-safety or logistics service.
- Concurrency guarantees rely on MySQL InnoDB row locking; the project must be tested on MySQL.
- AI recommendations depend on the quality of user-entered data; external AI calls may incur cost or latency.

## 18. Out of Scope and Future Enhancements

- Live GPS tracking and route optimisation; third-party courier scheduling.
- Online payments, tax receipts and financial transactions.
- Native Android and iOS applications; multilingual interface; voice assistant.
- Demand forecasting with machine learning; automated image-based quality or spoilage detection.
- Integration with NGO registries or government platforms; beneficiary accounts and individual beneficiary records.

## 19. Glossary

| Term | Definition |
|---|---|
| **API** | Application Programming Interface. |
| **Allocation** | Accepted quantity assigned to one request from one donation. |
| **CHECK constraint** | A database rule that rejects rows violating a condition (for example reserved greater than total). |
| **Optimistic locking** | Detecting conflicting edits through a version number instead of blocking. |
| **Row-level lock** | A database lock on one record (select_for_update) that makes concurrent actions wait in turn. |
| **DRF** | Django REST Framework. |
| **MVP** | Minimum Viable Product: the smallest release that delivers the core workflow correctly. |
| **RBAC** | Role-Based Access Control. |
| **Human-in-the-loop** | A person reviews or approves a system or AI suggestion before action. |
| **SRS** | Software Requirements Specification. |

**End of Document.** This SRS is a proposed baseline for discussion and instructor approval. Requirements may be refined after review, especially account verification, food-safety procedures, location handling and the selected AI provider.